# Assignment6 solution

## 1. ALSU UVM test

- **Top module**

```
import uvm_pkg::*;
`include "uvm_macros.svh"

import alsu_test_pkg::*;
module top();
    //? clk generation
    bit clk;
    always #1 clk = ~clk;
    //? DUT & interface instantiation
    ALSU_if alsuif (clk);
    ALSU DUT (
        .clk(alsuif.clk),
        .cin(alsuif.cin),
        .rst(alsuif.rst),
        .red_op_A(alsuif.red_op_A),
        .red_op_B(alsuif.red_op_B),
        .bypass_A(alsuif.bypass_A),
        .bypass_B(alsuif.bypass_B),
        .direction(alsuif.direction),
        .serial_in(alsuif.serial_in),
        .opcode(alsuif.opcode),
        .A(alsuif.A),
        .B(alsuif.B),
        .leds(alsuif.leds),
        .out(alsuif.out)
    );
    //? UVM run_test task
    initial begin
        uvm_config_db #(virtual ALSU_if)::set(null, "uvm_test_top", "ALSU_IF", alsuif);
        run_test("alsu_test");
    end

    bind ALSU alsu_sva alsu_sva_inst(alsuif.MONITOR);

endmodule
```

- **Interface**

```
import shared_pkg::*;
interface ALSU_if (clk);
  input clk;
  logic cin, rst, red_op_A, red_op_B, bypass_A, bypass_B, direction, serial_in;
  opcode_e opcode;
  logic signed [2:0] A, B;
  logic [15:0] leds;
  logic signed [5:0] out;

  modport MONITOR(
      input clk,
            cin,
            rst,
            red_op_A,
            red_op_B,
            bypass_A,
            bypass_B,
            direction,
            serial_in,
            opcode,
            A,
            B,
            leds,
            out
  );
endinterface
```

- **Shared package**

```
package shared_pkg;
  parameter INPUT_PRIORITY = "A";
  parameter FULL_ADDER = "ON";
  parameter MAXPOS = 3'b011;
  parameter MAXNEG = 3'b100;
  parameter ZERO = 3'b000;
  typedef enum logic [2:0] {
    OR,
    XOR,
    ADD,
    MULT,
    SHIFT,
    ROTATE,
    INVALID_6,
    INVALID_7
  } opcode_e;
endpackage
```

- **UVM test**

```
package alsu_test_pkg;
    import uvm_pkg::*;
    import alsu_env_pkg::*;
    import alsu_config_obj_pkg::*;
    import alsu_seq_pkg::*;
    `include "uvm_macros.svh"

    class alsu_test extends uvm_test;
        `uvm_component_utils(alsu_test)

        alsu_env env;
        alsu_config_obj alsu_cfg;
        alsu_main_seq main_seq;
        alsu_reset_seq reset_seq;


        //? constructor
        function new(string name="alsu_test", uvm_component parent = null);
            super.new(name, parent);
        endfunction

        //? phases
        function void build_phase(uvm_phase phase);
            super.build_phase(phase);
            env = alsu_env::type_id::create("env",this);
            alsu_cfg = alsu_config_obj::type_id::create("alsu_cfg");
            main_seq = alsu_main_seq::type_id::create("main_seq");
            reset_seq = alsu_reset_seq::type_id::create("reset_seq");

            if(!uvm_config_db #(virtual ALSU_if)::get(this,"","ALSU_IF", alsu_cfg.alsu_vif))
                `uvm_fatal("build_phase", "TEST - unable to get virtual interface")

            uvm_config_db #(alsu_config_obj)::set(this,"*","CFG",alsu_cfg);
        endfunction

        task run_phase(uvm_phase phase);
            super.run_phase(phase);
            phase.raise_objection(this);
            `uvm_info("run_phase", "Reset asserted", UVM_LOW)
            reset_seq.start(env.agt.sqr);
            `uvm_info("run_phase", "Reset deasserted", UVM_LOW)

            `uvm_info("run_phase", "main stimulus generation started", UVM_LOW)
            main_seq.start(env.agt.sqr);
            `uvm_info("run_phase", "main stimulus generation ended", UVM_LOW)
            phase.drop_objection(this);
        endtask

    endclass
endpackage
```

- **UVM sequence item**

```
package alsu_seq_item_pkg;
    import uvm_pkg::*;
    import shared_pkg::*;
    `include "uvm_macros.svh"

    class alsu_seq_item extends uvm_sequence_item;
        `uvm_object_utils(alsu_seq_item)

        rand bit cin, rst, red_op_A, red_op_B, bypass_A, bypass_B, direction, serial_in;
        rand opcode_e opcode;
        rand bit [2:0] A,B;
        logic [15:0] leds;
        logic [5:0] out;


        function new(string name = "alsu_seq_item");
            super.new(name);
        endfunction

        constraint alsu_constraints {
            rst dist {0:=99, 1:=1};
            bypass_A dist {0:=99, 1:=1};
            bypass_B dist {0:=99, 1:=1};
            opcode dist {INVALID_6:=10, INVALID_7:=10, [OR:ROTATE]:=90};
            if(opcode == OR || opcode == XOR){
                red_op_A dist {1:=30, 0:=70};
                red_op_B dist {1:=30, 0:=70};
                if(red_op_A) {
                    $onehot(A);
                    B == 0;
                } else if(red_op_B){
                    $onehot(B);
                    A == 0;
                }
            } else if(opcode == ADD || opcode == MULT) {
                A dist {MAXPOS:=80, MAXNEG:=80, ZERO:=80, {3'd1,3'd2,3'd5,3'd6,3'd7}:=20};
                B dist {MAXPOS:=80, MAXNEG:=80, ZERO:=80, {3'd1,3'd2,3'd5,3'd6,3'd7}:=20};
                red_op_A dist {1:=1, 0:=99};
                red_op_B dist {1:=1, 0:=99};
            } else {
                red_op_A dist {1:=1, 0:=99};
                red_op_B dist {1:=1, 0:=99};
            }
        }

        function string convert2string();
            return $sformatf("%s reset = 0b%0b, red_op_A = 0b%0b, red_op_B = %s, bypass_A = %s, bypass_B = 0b%0b, direction = 0b%0b, serial_in = 0b%0b, opcode = %s, A = 0b%0b, B = 0b%0b",
            super.convert2string(), rst, red_op_A, red_op_B, bypass_A, bypass_B, direction, serial_in, opcode.name(), A, B);
        endfunction

        function string convert2string_stimulus();
            return $sformatf("reset = 0b%0b, red_op_A = 0b%0b, red_op_B = %s, bypass_A = %s, bypass_B = 0b%0b, direction = 0b%0b, serial_in = 0b%0b, opcode = %s, A = 0b%0b, B = 0b%0b",
            rst, red_op_A, red_op_B, bypass_A, bypass_B, direction, serial_in, opcode.name(), A, B);
        endfunction

    endclass //alsu_seq_item extends superClass
endpackage
```

- **UVM sequence**

```
package  alsu_seq_pkg;
    import uvm_pkg::*;
    import shared_pkg::*;
    import alsu_seq_item_pkg::*;
    `include "uvm_macros.svh"

    class alsu_reset_seq extends uvm_sequence #(alsu_seq_item);
        `uvm_object_utils(alsu_reset_seq)
        alsu_seq_item seq_item;

        function new(string name = "alsu_seq");
            super.new(name);
        endfunction

        task body;
            seq_item = alsu_seq_item::type_id::create("seq_item");
            start_item(seq_item);
            seq_item.rst = 1;
            seq_item.cin = 0;
            seq_item.serial_in = 0;
            seq_item.direction = 0;
            seq_item.red_op_A = 0;
            seq_item.red_op_B = 0;
            seq_item.bypass_A = 0;
            seq_item.bypass_B = 0;
            seq_item.opcode = opcode_e'($random);
            seq_item.A = $random;
            seq_item.B = $random;
            finish_item(seq_item);
        endtask
    endclass //shift_reg_seq extends superClass

    class alsu_main_seq extends uvm_sequence #(alsu_seq_item);
        `uvm_object_utils(alsu_main_seq)
        alsu_seq_item seq_item;

        function new(string name = "alsu_seq_item");
            super.new(name);
        endfunction

        task body;
            seq_item = alsu_seq_item::type_id::create("seq_item");
            repeat(10000) begin
                start_item(seq_item);
                assert(seq_item.randomize());
                finish_item(seq_item);
            end
        endtask
    endclass //shift_reg_seq extends superClass


endpackage
```

- **UVM sequencer**

```
package alsu_sqr_pkg;
  import uvm_pkg::*;
  import alsu_seq_item_pkg::*;
  `include "uvm_macros.svh"

  class alsu_sqr extends uvm_sequencer #(alsu_seq_item);
    `uvm_component_utils(alsu_sqr)

    function new(string name = "alsu_sqr", uvm_component parent = null);
      super.new(name, parent);
    endfunction

  endclass

endpackage
```

- **UVM environment**

```
package alsu_env_pkg;
    import uvm_pkg::*;
    import alsu_cover_pkg::*;
    import alsu_agent_pkg::*;
    `include "uvm_macros.svh"

    class alsu_env extends uvm_env;
        `uvm_component_utils(alsu_env)

        alsu_agent agt;
        alsu_cover cov;

        function new(string name="alsu_env", uvm_component parent = null);
            super.new(name, parent);
        endfunction

        function void build_phase(uvm_phase phase);
            super.build_phase(phase);
            agt = alsu_agent::type_id::create("agt",this);
            cov = alsu_cover::type_id::create("cov",this);
        endfunction

        function void connect_phase(uvm_phase phase);
            super.connect_phase(phase);
            agt.agt_ap.connect(cov.cov_export);
        endfunction
    endclass
endpackage
```

- **UVM coverage collector**

```
package alsu_cover_pkg;
  import uvm_pkg::*;
  `include "uvm_macros.svh"
  import alsu_seq_item_pkg::*;
  import shared_pkg::*;

  class alsu_cover extends uvm_component;
    `uvm_component_utils(alsu_cover)
    uvm_analysis_export #(alsu_seq_item) cov_export;
    uvm_tlm_analysis_fifo #(alsu_seq_item) cov_fifo;
    alsu_seq_item seq_item_cov;

    covergroup cg;
        A_cp: coverpoint seq_item_cov.A {
                bins A_data_0 = {0};
                bins A_data_max = {MAXPOS};
                bins A_data_min = {MAXNEG};
                bins A_data_default = default;
            }
            A_data_cp: coverpoint seq_item_cov.A iff (seq_item_cov.red_op_A) {
                bins A_data_walkingones[] = {3'b001, 3'b010, 3'b100};
            }
            B_cp: coverpoint seq_item_cov.B {
                bins B_data_0 = {0};
                bins B_data_max = {MAXPOS};
                bins B_data_min = {MAXNEG};
                bins B_data_default = default;
            }
            B_data_cp: coverpoint seq_item_cov.B iff (seq_item_cov.red_op_B && !seq_item_cov.red_op_A){
                bins B_data_walkingones[] = {3'b001, 3'b010, 3'b100};
            }
            opcode_cp: coverpoint seq_item_cov.opcode {
                bins Bins_shift[] = {SHIFT,ROTATE};
                bins Bins_arith[] = {ADD,MULT};
                bins Bins_bitwise[] = {OR,XOR};
                illegal_bins Bins_invalid = {INVALID_6,INVALID_7};
            }
            cin_cp: coverpoint seq_item_cov.cin iff (seq_item_cov.opcode == ADD);
            direction_cp: coverpoint seq_item_cov.direction iff(seq_item_cov.opcode inside {SHIFT, ROTATE});
            serial_in_cp: coverpoint seq_item_cov.serial_in iff(seq_item_cov.opcode == SHIFT);
            red_opA_cp: coverpoint seq_item_cov.red_op_A {option.weight = 0;}
            red_opB_cp: coverpoint seq_item_cov.red_op_B {
                option.weight = 0;
                bins red_op_B0 = {0};
                bins red_op_B1 = {1};
            }
            ALU_cross_1_cp: cross A_cp, B_cp, opcode_cp {
                option.cross_auto_bin_max = 0;
                bins A_B_cross = binsof(A_cp) && binsof(B_cp) && binsof(opcode_cp.Bins_arith);
            }
            ALU_cross_2_cp: cross A_data_cp, B_cp iff(seq_item_cov.opcode inside {OR,XOR}){
                option.cross_auto_bin_max = 0;
                bins A_walking_cross_B = binsof(A_data_cp) && binsof(B_cp.B_data_0);
            }
            ALU_cross_3_cp: cross B_data_cp, A_cp, opcode_cp{
                option.cross_auto_bin_max = 0;
                bins B_walking_cross_A = binsof(B_data_cp) && binsof(A_cp.A_data_0) && binsof(opcode_cp) intersect {OR,XOR};
            }
```

```
            ALU_cross_4_cp: cross red_opA_cp, red_opB_cp iff (seq_item_cov.opcode != OR && seq_item_cov.opcode != XOR) {
                option.cross_auto_bin_max = 0;
                bins invalid_case = binsof(red_opA_cp) intersect {1} || binsof(red_opB_cp.red_op_B1);
            }
        endgroup

    function new(string name = "alsu_cover", uvm_component parent = null);
        super.new(name, parent);
        cg = new();
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        cov_export = new("cov_export", this);
        cov_fifo = new("cov_fifo", this);
    endfunction

    function void connect_phase(uvm_phase phase);
        super.build_phase(phase);
        cov_export.connect(cov_fifo.analysis_export);
    endfunction

    task run_phase(uvm_phase phase);
        super.run_phase(phase);
        forever begin
            cov_fifo.get(seq_item_cov);
            cg.sample();
        end
    endtask
  endclass
endpackage
```

- **UVM agent**

```
package alsu_agent_pkg;
  import uvm_pkg::*;
  `include "uvm_macros.svh"
  import alsu_seq_item_pkg::*;
  import alsu_sqr_pkg::*;
  import alsu_driver_pkg::*;
  import alsu_monitor_pkg::*;
  import alsu_config_obj_pkg::*;

  class alsu_agent extends uvm_agent;
    `uvm_component_utils(alsu_agent)
    alsu_sqr sqr;
    alsu_driver drv;
    alsu_monitor mon;
    alsu_config_obj alsu_cfg;
    uvm_analysis_port #(alsu_seq_item) agt_ap;

    function new(string name = "alsu_agent", uvm_component parent = null);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        sqr = alsu_sqr::type_id::create("sqr", this);
        drv = alsu_driver::type_id::create("drv", this);
        mon = alsu_monitor::type_id::create("mon", this);
        alsu_cfg = alsu_config_obj::type_id::create("alsu_cfg");
        agt_ap = new("agt_ap", this);
        if(!uvm_config_db #(alsu_config_obj)::get(this,"","CFG",alsu_cfg)) begin
            `uvm_fatal("build_phase", "TEST - unable to get configuration object")
        end
    endfunction


    function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
        drv.alsu_vif = alsu_cfg.alsu_vif;
        mon.alsu_vif = alsu_cfg.alsu_vif;
        drv.seq_item_port.connect(sqr.seq_item_export);
        mon.mon_ap.connect(agt_ap);
    endfunction
  endclass

endpackage
```

- **UVM driver**

```
package alsu_driver_pkg;
  import uvm_pkg::*;
  `include "uvm_macros.svh"
  import alsu_seq_item_pkg::*;

  class alsu_driver extends uvm_driver #(alsu_seq_item);
    `uvm_component_utils(alsu_driver)
    virtual ALSU_if alsu_vif;
    alsu_seq_item stim_seq_item;

    function new(string name = "alsu_driver", uvm_component parent = null);
        super.new(name,parent);
    endfunction

    task run_phase(uvm_phase phase);
      super.run_phase(phase);
      stim_seq_item = alsu_seq_item::type_id::create("stim_seq_item");
      forever begin
        seq_item_port.get_next_item(stim_seq_item);
        alsu_vif.rst = stim_seq_item.rst;
        alsu_vif.cin = stim_seq_item.cin;
        alsu_vif.serial_in = stim_seq_item.serial_in;
        alsu_vif.direction = stim_seq_item.direction;
        alsu_vif.red_op_A = stim_seq_item.red_op_A;
        alsu_vif.red_op_B = stim_seq_item.red_op_B;
        alsu_vif.bypass_A = stim_seq_item.bypass_A;
        alsu_vif.bypass_B = stim_seq_item.bypass_B;
        alsu_vif.opcode = stim_seq_item.opcode;
        alsu_vif.A = stim_seq_item.A;
        alsu_vif.B = stim_seq_item.B;
        @(negedge alsu_vif.clk);
        @(negedge alsu_vif.clk);
        seq_item_port.item_done();
        `uvm_info("run_phase", stim_seq_item.convert2string_stimulus(), UVM_HIGH)
      end
    endtask
  endclass

endpackage
```

- **UVM monitor**

```
package alsu_monitor_pkg;
  import uvm_pkg::*;
  `include "uvm_macros.svh"
  import alsu_seq_item_pkg::*;

  class alsu_monitor extends uvm_monitor;
    `uvm_component_utils(alsu_monitor)
    virtual ALSU_if alsu_vif;
    alsu_seq_item rsp_seq_item;
    uvm_analysis_port #(alsu_seq_item) mon_ap;

    function new(string name = "alsu_monitor", uvm_component parent = null);
        super.new(name, parent);
    endfunction

    // Build the driver in the build phase
  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    mon_ap = new("mon_ap", this);
  endfunction

    task run_phase(uvm_phase phase);
      super.run_phase(phase);
      forever begin
        rsp_seq_item = alsu_seq_item::type_id::create("rsp_seq_item");
        @(negedge alsu_vif.clk);
        @(negedge alsu_vif.clk);
        rsp_seq_item.rst = alsu_vif.rst;
        rsp_seq_item.cin = alsu_vif.cin;
        rsp_seq_item.serial_in = alsu_vif.serial_in;
        rsp_seq_item.direction = alsu_vif.direction;
        rsp_seq_item.red_op_A = alsu_vif.red_op_A;
        rsp_seq_item.red_op_B = alsu_vif.red_op_B;
        rsp_seq_item.bypass_A = alsu_vif.bypass_A;
        rsp_seq_item.bypass_B = alsu_vif.bypass_B;
        rsp_seq_item.opcode = alsu_vif.opcode;
        rsp_seq_item.A = alsu_vif.A;
        rsp_seq_item.B = alsu_vif.B;
        rsp_seq_item.out = alsu_vif.out;
        rsp_seq_item.leds = alsu_vif.leds;
        mon_ap.write(rsp_seq_item);
        `uvm_info("run_phase", rsp_seq_item.convert2string(), UVM_HIGH)
      end
    endtask
  endclass

endpackage
```

- **Configuration object**

```
package alsu_config_obj_pkg;
    import uvm_pkg::*;
    `include "uvm_macros.svh"

    class alsu_config_obj extends uvm_object;
        `uvm_object_utils(alsu_config_obj)

        virtual ALSU_if alsu_vif;

        function new(string name = "alsu_config_obj");
            super.new(name);
        endfunction

    endclass
endpackage
```

- **SVA file**

```
module alsu_sva(ALSU_if.MONITOR alsuif);
    import shared_pkg::*;
    bit invalid, normal_op;
    int correct_count, error_count;
    //? asynchronous reset assertion
    always_comb begin
        if(alsuif.rst)
            arst_sva: assert final(alsuif.leds == 0 && alsuif.out == 0);
    end
    assign invalid = ( (alsuif.red_op_A | alsuif.red_op_B) & (alsuif.opcode[1] | alsuif.opcode[2]) ) | (alsuif.opcode[1] & alsuif.opcode[2]);
    assign normal_op = ~alsuif.bypass_A & ~alsuif.bypass_B & ~invalid;
    //? PROPERTIES
    property invalid_leds_sva_check;
        @(posedge alsuif.clk) disable iff(alsuif.rst) invalid && !alsuif.bypass_A && !alsuif.bypass_B |=>
                                                      ##1 alsuif.leds == ~$past(alsuif.leds);
    endproperty

    property bypassA_sva_check;
        @(posedge alsuif.clk) disable iff(alsuif.rst) alsuif.bypass_A |=>
                                                      ##1 alsuif.out == $past(alsuif.A,2);
    endproperty

    property bypassB_sva_check;
        @(posedge alsuif.clk) disable iff(alsuif.rst) alsuif.bypass_B && !alsuif.bypass_A |=>
                                                      ##1 alsuif.out == $past(alsuif.B,2);
    endproperty

    property invalid_out_sva_check;
        @(posedge alsuif.clk) disable iff(alsuif.rst) !alsuif.bypass_A && !alsuif.bypass_B && invalid |=> ##1 alsuif.out == 0;
    endproperty

    property red_ora_sva_check;
        @(posedge alsuif.clk) disable iff(alsuif.rst) normal_op && alsuif.opcode == OR && alsuif.red_op_A |=>
                                                      ##1 alsuif.out == |$past(alsuif.A,2);
    endproperty

    property red_orb_sva_check;
        @(posedge alsuif.clk) disable iff(alsuif.rst) normal_op && alsuif.opcode == OR && alsuif.red_op_B && !alsuif.red_op_A |=>
                                                      ##1 alsuif.out == |$past(alsuif.B,2);
    endproperty

    property btw_or_sva_check;
        @(posedge alsuif.clk) disable iff(alsuif.rst) normal_op && alsuif.opcode == OR && !alsuif.red_op_A && !alsuif.red_op_B |=>
                                                      ##1 alsuif.out == $past(alsuif.A,2) | $past(alsuif.B,2);
    endproperty

    property red_xora_sva_check;
        @(posedge alsuif.clk) disable iff(alsuif.rst) normal_op && alsuif.opcode == XOR && alsuif.red_op_A |=>
                                                      ##1 alsuif.out == ^$past(alsuif.A,2);
    endproperty
```

```
    property red_xorb_sva_check;
        @(posedge alsuif.clk) disable iff(alsuif.rst) normal_op && alsuif.opcode == XOR && alsuif.red_op_B && !alsuif.red_op_A |=>
                                                ##1 alsuif.out == ^$past(alsuif.B,2);
    endproperty

    property btw_xor_sva_check;
        @(posedge alsuif.clk) disable iff(alsuif.rst) normal_op && alsuif.opcode == XOR && !alsuif.red_op_A && !alsuif.red_op_B |=>
                                                ##1 alsuif.out == $past(alsuif.A,2) ^ $past(alsuif.B,2);
    endproperty

    property add_sva_check;
        @(posedge alsuif.clk) disable iff(alsuif.rst) normal_op && alsuif.opcode == ADD |=>
                                                ##1 alsuif.out == $signed(6'($past(alsuif.A, 2))) + $signed(6'($past(alsuif.B, 2))) + $past(alsuif.cin, 2);
    endproperty

    property mult_sva_check;
        @(posedge alsuif.clk) disable iff(alsuif.rst) normal_op && alsuif.opcode == MULT |=>
                                                ##1 alsuif.out == $past(alsuif.A,2) * $past(alsuif.B,2);
    endproperty

    property shift_left_sva_check;
        @(posedge alsuif.clk) disable iff(alsuif.rst) normal_op && alsuif.opcode == SHIFT && alsuif.direction |=>
                                                ##1 alsuif.out == {$past(alsuif.out[4:0]), $past(alsuif.serial_in,2)};
    endproperty

    property shift_right_sva_check;
        @(posedge alsuif.clk) disable iff(alsuif.rst) normal_op && alsuif.opcode == SHIFT && !alsuif.direction|=>
                                                ##1 alsuif.out == {$past(alsuif.serial_in,2), $past(alsuif.out[5:1])};
    endproperty

    property rotate_left_sva_check;
        @(posedge alsuif.clk) disable iff(alsuif.rst) normal_op && alsuif.opcode == ROTATE && alsuif.direction|=>
                                                ##1 alsuif.out == {$past(alsuif.out[4:0]), $past(alsuif.out[5])};
    endproperty

    property rotate_right_sva_check;
        @(posedge alsuif.clk) disable iff(alsuif.rst) normal_op && alsuif.opcode == ROTATE && !alsuif.direction|=>
                                                ##1 alsuif.out == {$past(alsuif.out[0]), $past(alsuif.out[5:1])};
    endproperty
```

```
    //? ASSERTIONS
    invalid_leds_sva:       assert property (invalid_leds_sva_check);
    bypassA_sva:            assert property (bypassA_sva_check);
    bypassB_sva:            assert property (bypassB_sva_check);
    invalid_out_sva:        assert property (invalid_out_sva_check);
    red_ora_sva:            assert property (red_ora_sva_check);
    red_orb_sva:            assert property (red_orb_sva_check);
    btw_or_sva:             assert property (btw_or_sva_check);
    red_xora_sva:           assert property (red_xora_sva_check);
    red_xorb_sva:           assert property (red_xorb_sva_check);
    add_sva:                assert property (add_sva_check);
    mult_sva:               assert property (mult_sva_check);
    shift_left_sva:         assert property (shift_left_sva_check);
    shift_right_sva:        assert property (shift_right_sva_check);
    rotate_left_sva:        assert property (rotate_left_sva_check);
    rotate_right_sva:       assert property (rotate_right_sva_check);
    //? COVER PROPERTIES
    invalid_leds_sva_cp:       cover property (invalid_leds_sva_check);
    bypassA_sva_cp:            cover property (bypassA_sva_check);
    bypassB_sva_cp:            cover property (bypassB_sva_check) ;
    invalid_out_sva_cp:        cover property (invalid_out_sva_check);
    red_ora_sva_cp:            cover property (red_ora_sva_check);
    red_orb_sva_cp:            cover property (red_orb_sva_check);
    btw_or_sva_cp:             cover property (btw_or_sva_check);
    red_xora_sva_cp:           cover property (red_xora_sva_check);
    red_xorb_sva_cp:           cover property (red_xorb_sva_check);
    add_sva_cp:                cover property (add_sva_check);
    mult_sva_cp:               cover property (mult_sva_check);
    shift_left_sva_cp:         cover property (shift_left_sva_check);
    shift_right_sva_cp:        cover property (shift_right_sva_check);
    rotate_left_sva_cp:        cover property (rotate_left_sva_check);
    rotate_right_sva_cp:       cover property (rotate_right_sva_check);

endmodule
```

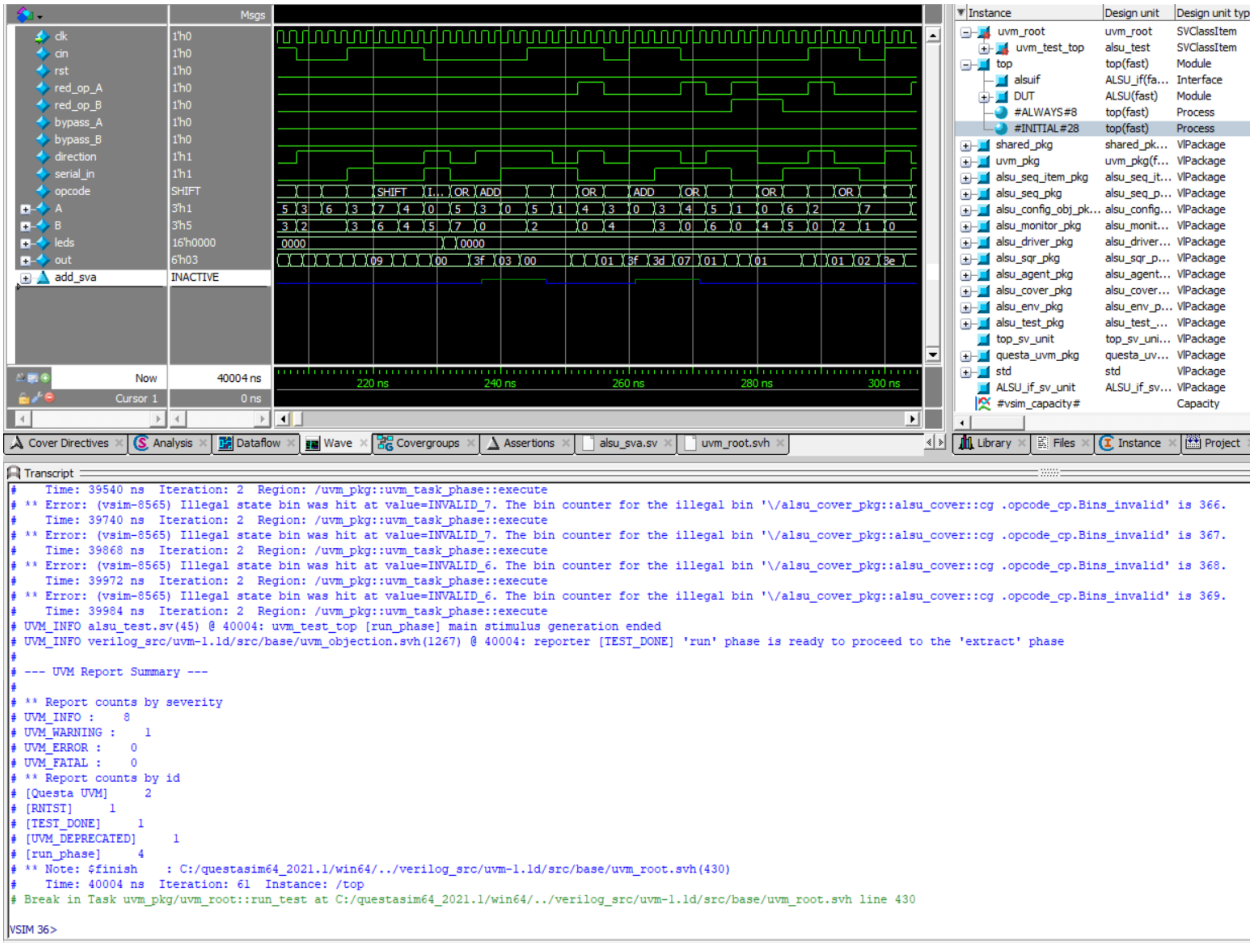
- **Src list**

```
ALSU.v
shared_pkg.sv
ALSU_if.sv
alsu_sva.sv
alsu_seq_item.sv
alsu_seq.sv
alsu_sqr.sv
alsu_config_obj.sv
alsu_driver.sv
alsu_monitor.sv
alsu_agent.sv
alsu_cover.sv
alsu_env.sv
alsu_test.sv
top.sv
```

- **Do file**

```
vlib work
vlog -f src_files.list +cover -covercells
vsim -voptargs=+acc work.top -classdebug -uvmcontrol=all -cover
coverage save alsu.ucdb -onexit
add wave /top/alsuif/*
add wave /top/DUT/alsu_sva_inst/add_sva
run -all
coverage exclude -du ALSU -togglenode {cin_reg[1]}
coverage exclude -src ALSU.v -line 76 -code b
quit -sim
vcover report alsu.ucdb -du=ALSU -details -annotate -all -output code_cover_rpt.txt
```

- **Simulation results**



- **Assertion results**

| Name | Assertion Type | Language | Enable | Failure Count | Pass Count | Active Count | Memory | Peak Memory |
|---|---|---|---|---|---|---|---|---|
| /uvm_pkg::uvm_reg_map::do_write/#ublk#215181159#1731/im... | Immediate | SVA | on | 0 | 0 | - | - | - |
| /uvm_pkg::uvm_reg_map::do_read/#ublk#215181159#1771/im... | Immediate | SVA | on | 0 | 0 | - | - | - |
| /alsu_seq_pkg::alsu_main_seq::body/#ublk#256298615#43/imm... | Immediate | SVA | on | 0 | 1 | - | - | - |
| /top/DUT/alsu_sva_inst/arst_sva | Immediate | SVA | on | 0 | 1 | - | - | - |
| /top/DUT/alsu_sva_inst/invalid_leds_sva | Concurrent | SVA | on | 0 | 1 | - | 0B | 0B |
| /top/DUT/alsu_sva_inst/bypassA_sva | Concurrent | SVA | on | 0 | 1 | - | 0B | 0B |
| /top/DUT/alsu_sva_inst/bypassB_sva | Concurrent | SVA | on | 0 | 1 | - | 0B | 0B |
| /top/DUT/alsu_sva_inst/invalid_out_sva | Concurrent | SVA | on | 0 | 1 | - | 0B | 0B |
| /top/DUT/alsu_sva_inst/red_ora_sva | Concurrent | SVA | on | 0 | 1 | - | 0B | 0B |
| /top/DUT/alsu_sva_inst/red_orb_sva | Concurrent | SVA | on | 0 | 1 | - | 0B | 0B |
| /top/DUT/alsu_sva_inst/btw_or_sva | Concurrent | SVA | on | 0 | 1 | - | 0B | 0B |
| /top/DUT/alsu_sva_inst/red_xora_sva | Concurrent | SVA | on | 0 | 1 | - | 0B | 0B |
| /top/DUT/alsu_sva_inst/red_xorb_sva | Concurrent | SVA | on | 0 | 1 | - | 0B | 0B |
| /top/DUT/alsu_sva_inst/add_sva | Concurrent | SVA | on | 0 | 1 | - | 0B | 0B |
| /top/DUT/alsu_sva_inst/mult_sva | Concurrent | SVA | on | 0 | 1 | - | 0B | 0B |
| /top/DUT/alsu_sva_inst/shift_left_sva | Concurrent | SVA | on | 0 | 1 | - | 0B | 0B |
| /top/DUT/alsu_sva_inst/shift_right_sva | Concurrent | SVA | on | 0 | 1 | - | 0B | 0B |
| /top/DUT/alsu_sva_inst/rotate_left_sva | Concurrent | SVA | on | 0 | 1 | - | 0B | 0B |
| /top/DUT/alsu_sva_inst/rotate_right_sva | Concurrent | SVA | on | 0 | 1 | - | 0B | 0B |

Cover Directives | Analysis | Dataflow | Wave | Covergroups | Assertions | alsu_sva.sv | uvm_root.svh

- **Assertion coverage**

| Name | Language | Enabled | Log | Count | AtLeast | Limit | Weight | Cmplt % | Cmplt graph | Includ |
|---|---|---|---|---|---|---|---|---|---|---|
| /top/DUT/alsu_sva_inst/invalid_leds_sva_cp | SVA | ✓ | Off | 968 | 1 | Unlimited | 1 | 100% | | ✓ |
| /top/DUT/alsu_sva_inst/bypassA_sva_cp | SVA | ✓ | Off | 204 | 1 | Unlimited | 1 | 100% | | ✓ |
| /top/DUT/alsu_sva_inst/bypassB_sva_cp | SVA | ✓ | Off | 172 | 1 | Unlimited | 1 | 100% | | ✓ |
| /top/DUT/alsu_sva_inst/invalid_out_sva_cp | SVA | ✓ | Off | 968 | 1 | Unlimited | 1 | 100% | | ✓ |
| /top/DUT/alsu_sva_inst/red_ora_sva_cp | SVA | ✓ | Off | 888 | 1 | Unlimited | 1 | 100% | | ✓ |
| /top/DUT/alsu_sva_inst/red_orb_sva_cp | SVA | ✓ | Off | 576 | 1 | Unlimited | 1 | 100% | | ✓ |
| /top/DUT/alsu_sva_inst/btw_or_sva_cp | SVA | ✓ | Off | 1436 | 1 | Unlimited | 1 | 100% | | ✓ |
| /top/DUT/alsu_sva_inst/red_xora_sva_cp | SVA | ✓ | Off | 956 | 1 | Unlimited | 1 | 100% | | ✓ |
| /top/DUT/alsu_sva_inst/red_xorb_sva_cp | SVA | ✓ | Off | 668 | 1 | Unlimited | 1 | 100% | | ✓ |
| /top/DUT/alsu_sva_inst/add_sva_cp | SVA | ✓ | Off | 3022 | 1 | Unlimited | 1 | 100% | | ✓ |
| /top/DUT/alsu_sva_inst/mult_sva_cp | SVA | ✓ | Off | 2984 | 1 | Unlimited | 1 | 100% | | ✓ |
| /top/DUT/alsu_sva_inst/shift_left_sva_cp | SVA | ✓ | Off | 1456 | 1 | Unlimited | 1 | 100% | | ✓ |
| /top/DUT/alsu_sva_inst/shift_right_sva_cp | SVA | ✓ | Off | 1622 | 1 | Unlimited | 1 | 100% | | ✓ |
| /top/DUT/alsu_sva_inst/rotate_left_sva_cp | SVA | ✓ | Off | 1490 | 1 | Unlimited | 1 | 100% | | ✓ |
| /top/DUT/alsu_sva_inst/rotate_right_sva_cp | SVA | ✓ | Off | 1536 | 1 | Unlimited | 1 | 100% | | ✓ |

- **Functional coverage**

| Name | Class Type | Coverage | Goal | % of Goal | Status | Included | Merge_instances | Get_inst_coverage |
|---|---|---|---|---|---|---|---|---|
| /alsu_cover_pkg/alsu_cover | | 100.00% | | | | | | |
| TYPE cg | | 100.00% | 100 | 100.00... | | ✓ | auto(1) | |
| CVP cg::A_cp | | 100.00% | 100 | 100.00... | | ✓ | | |
| CVP cg::A_data_cp | | 100.00% | 100 | 100.00... | | ✓ | | |
| CVP cg::B_cp | | 100.00% | 100 | 100.00... | | ✓ | | |
| CVP cg::B_data_cp | | 100.00% | 100 | 100.00... | | ✓ | | |
| CVP cg::opcode_cp | | 100.00% | 100 | 100.00... | | ✓ | | |
| CVP cg::cin_cp | | 100.00% | 100 | 100.00... | | ✓ | | |
| CVP cg::direction_cp | | 100.00% | 100 | 100.00... | | ✓ | | |
| CVP cg::serial_in_cp | | 100.00% | 100 | 100.00... | | ✓ | | |
| CVP cg::red_opA_cp | | 100.00% | 100 | 100.00... | | ✓ | | |
| CVP cg::red_opB_cp | | 100.00% | 100 | 100.00... | | ✓ | | |
| CROSS cg::ALU_cross_1_cp | | 100.00% | 100 | 100.00... | | ✓ | | |
| CROSS cg::ALU_cross_2_cp | | 100.00% | 100 | 100.00... | | ✓ | | |
| CROSS cg::ALU_cross_3_cp | | 100.00% | 100 | 100.00... | | ✓ | | |
| CROSS cg::ALU_cross_4_cp | | 100.00% | 100 | 100.00... | | ✓ | | |

Cover Directives | Analysis | Dataflow | Wave | Covergroups | Assertions | alsu_sva.sv | uvm_root.svh

- ## Code coverage

### ❖ Branch

```
================================================================================
=== Design Unit: work.ALSU
================================================================================
Branch Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                        25        25         0   100.00%

================================Branch Details================================

Branch Coverage for Design Unit work.ALSU
```

### ❖ Statement

```
Statement Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Statements                      48        48         0   100.00%

================================Statement Details================================

Statement Coverage for Design Unit work.ALSU --
```

### ❖ Toggle coverage

```
Toggle Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Toggles                        118       118         0   100.00%

================================Toggle Details================================

Toggle Coverage for Design Unit work.ALSU

                                    Node      1H->0L      0L->1H                    "Coverage"
                                    ----------------------------------------------------------
                                   A[0-2]           1           1                        100.00
                               A_reg[0-2]           1           1                        100.00
                                   B[0-2]           1           1                        100.00
                               B_reg[0-2]           1           1                        100.00
                                  bypass_A           1           1                        100.00
                             bypass_A_reg           1           1                        100.00
                                  bypass_B           1           1                        100.00
                             bypass_B_reg           1           1                        100.00
                                      cin           1           1                        100.00
                                 cin_reg[0]           1           1                        100.00
                                      clk           1           1                        100.00
                                 direction           1           1                        100.00
                             direction_reg           1           1                        100.00
                                   invalid           1           1                        100.00
                            invalid_opcode           1           1                        100.00
                            invalid_red_op           1           1                        100.00
                                leds[0-15]           1           1                        100.00
                               opcode[0-2]           1           1                        100.00
                            opcode_reg[0-2]           1           1                        100.00
                                   out[0-5]           1           1                        100.00
                                  red_op_A           1           1                        100.00
                              red_op_A_reg           1           1                        100.00
                                  red_op_B           1           1                        100.00
                              red_op_B_reg           1           1                        100.00
                                       rst           1           1                        100.00
                                 serial_in           1           1                        100.00
                             serial_in_reg           1           1                        100.00

Total Node Count     =         59
Toggled Node Count   =         59
Untoggled Node Count =          0

Toggle Coverage      =     100.00% (118 of 118 bins)


Total Coverage By Design Unit (filtered view): 100.00%
```